

End-to-End ML Versioning with Git, DVC & TensorFlow

Student: Umer

Repository: github.com/Umer529/git_dvc_assignment

Model: Fully connected ANN (not CNN)

Assignment: 3 (50 marks)

Dataset: Fashion-MNIST

Final tag: v2

Implementation summary

Four independent stages

- prepare.py** downloads and saves raw NumPy arrays.
- preprocess.py** normalizes and stratifies train/validation data.
- train.py** builds Flatten → Dense/ReLU → Dropout → Dense/Softmax.
- evaluate.py** writes JSON metrics and a confusion matrix.

Reproducibility

- All hyperparameters come from **params.yaml**.
- All four stages are wired through **dvc.yaml**.
- Raw data, processed data, model, history and plot are DVC cached.
- dvc.lock** records every dependency/output hash.

Part A — Git history and log variants

```
PS> git log --oneline --graph --all
* 2b51ccd (main) fix: restore pipeline output tracking after conflict demo
* 40120aa merge: reconcile normalization and DVC data conflict
| \
| * 19ffc5 (teammate-sim) experiment: teammate normalization and processed data
* | 457411b experiment: round main processed representation
| /
* 9e3fac9 (tag: v2, dev) experiment: increase hidden units for v2
* 5c2368d (tag: v1) feat: record reproducible v1 pipeline outputs
* f3f9a81 feat: define reproducible four-stage DVC pipeline
* 35eb3d9 chore: initialize DVC metadata
* 7aabd1b docs: note DVC status workflow
* b52b4a1 refactor: reorganize labels with git mv and remove obsolete notes
* b770404 docs: clarify preprocessing normalization
* dc2d965 feat: add metrics and confusion matrix evaluation
* 7f1db0e feat: add configurable ANN training stage
* e58a8f5 feat: add deterministic preprocessing stage
* 5f8c7f9 config: centralize pipeline hyperparameters
* 6284b16 feat: add Fashion-MNIST preparation stage
* b008809 docs: add project setup and dependency guide
* 2d50b09 fix: clarify project purpose in README
* 4a2a12c First Commit
```

Command	What it reveals beyond plain git log
git log --oneline --graph --all	Compact topology of every branch, merge, and tag.
git log --stat -3	Files changed and insertion/deletion totals for the latest three commits.
git log -p -1	The exact patch introduced by the latest commit.
git log main..dev	Commits reachable only from dev; after the final merge it is empty, proving main contains dev.

Git command demonstrations

A4 — Diff variants

```
PS> git diff
- # float32 keeps storage compact.
+ # Scale uint8 pixels to [0, 1] ...

PS> git add README.md
PS> git diff --staged
+ Run `dvc status` to check outputs.
```

```
PS> git diff main..dev --stat
9 files changed, 284 insertions(+), 1 deletion(-)
PS> git diff main...dev --stat
9 files changed, 284 insertions(+), 1 deletion(-)
```

Two-dot compares the two branch tips. **Three-dot** compares dev with the branches' merge base, isolating work introduced on dev. They matched here because main was an ancestor after rebase.

A5 — Stash scenario

```
PS> git stash push -m "preprocess normalization note"
Saved working directory ...
PS> git switch main; git switch dev
PS> git stash list
stash@{0}: On dev: preprocess normalization note
PS> git stash pop
Dropped refs/stash@{0}; src/preprocess.py restored
```

A7 — Reset scenario

```
PS> git reset --soft HEAD~1
M scratch_reset.txt # still staged
PS> git commit -m "restore second reset demo commit"
PS> git reset --hard HEAD~1
HEAD is now at a9bb0a0 # commit and change discarded
```

Soft reset moved HEAD but retained the index. Hard reset moved HEAD and restored both index and working tree.

A6 — Hotfix and rebase

```
BEFORE
* 2d50b09 (main, hotfix-readme) README fix
| * b4dc578 (dev) evaluation stage
| * ... six incremental dev commits
|/
* 4a2a12c First Commit

PS> git switch dev; git rebase main
CONFLICT (content): Merge conflict in README.md
Successfully rebased and updated refs/heads/dev.

AFTER
* dc2d965 (dev) evaluation stage
* ... rebased dev commits
* 2d50b09 (main) README fix
* 4a2a12c First Commit
```

The README conflict was manually reconciled; dev then sat linearly on top of the merged hotfix.

A8 — Tracked reorganization

```
PS> git mv labels.py src/labels.py
PS> git rm obsolete_notes.txt
PS> git show --stat --find-renames
delete mode 100644 obsolete_notes.txt
rename labels.py => src/labels.py (100%)
```

Parts B and C checks

- Environment: Python 3.12, TensorFlow 2.21, DVC 3.67.1.
- Installed Google Drive extension: dvc-gdrive 3.0.1.
- dvc init committed .dvc/config, .dvc/.gitignore, and .dvcignore.
- Credential and token patterns are excluded; OAuth client values belong in .dvc/config.local.

```
PS> dvc status
Data and pipelines are up to date.
PS> dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'evaluate' didn't change, skipping
```

DVC pipeline, parameters and experiment results

Final params.yaml

```
preprocess:
  validation_size: 0.2
  seed: 42
train:
  dense_units: 384
  dropout_rate: 0.2
  learning_rate: 0.001
  epochs: 10
  batch_size: 128
  seed: 42
```

D3 — First complete reproduction

```
Running stage 'prepare':
Saved raw data; train=(60000, 28, 28), test=(10000, 28, 28)
Generating lock file 'dvc.lock'
Running stage 'preprocess':
train=(48000, 28, 28), val=(12000, 28, 28)
Running stage 'train':
Epoch 10/10 ... val_accuracy: 0.8888
Running stage 'evaluate':
test_accuracy: 0.87380; test_loss: 0.34817
```

D4 — Parameter-aware rerun

```
Changed train.dense_units: 256 → 384
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train' ...
Running stage 'evaluate' ...
test_accuracy: 0.87840; test_loss: 0.33870
```

Only train depended on the changed key; evaluate then reran because the model hash changed. Upstream stages remained cached.

v1 versus v2 metrics

Version	Dense units	Test accuracy	Test loss	Macro F1
v1	256	0.87380	0.34817	0.87387
v2	384	0.87840	0.33870	0.87831

Result: 87.84% test accuracy, exceeding the required 85% target.

Final dvc.yaml

```
stages:
  prepare:
    cmd: python src/prepare.py
    deps: [src/prepare.py]
    outs: [data/raw]
  preprocess:
    cmd: python src/preprocess.py
    deps: [src/preprocess.py, data/raw]
    params:
      - preprocess.validation_size
      - preprocess.seed
    outs: [data/processed]
  train:
    cmd: python src/train.py
    deps: [src/train.py, data/processed]
    params:
      - train.dense_units
      - train.dropout_rate
      - train.learning_rate
      - train.epochs
      - train.batch_size
      - train.seed
    outs:
      - models/model.h5
      - models/history.csv
  evaluate:
    cmd: python src/evaluate.py
    deps:
      - src/evaluate.py
      - src/labels.py
      - models/model.h5
      - data/processed/x_test.npy
      - data/processed/y_test.npy
    outs: [models/confusion_matrix.png]
    metrics:
      - metrics.json:
          cache: false
```

Simulated collaboration and conflict resolution

E1–E3 — Independent code and data versions

teammate-sim
float64 normalization
DVC hash: 9e288ef9...dir
439,110,768 bytes

main
rounded float32 normalization
DVC hash: 557fc53f...dir
219,590,768 bytes

```
PS> git merge teammate-sim
Auto-merging data/processed.dvc
CONFLICT (content): Merge conflict in data/processed.dvc
Auto-merging src/preprocess.py
CONFLICT (content): Merge conflict in src/preprocess.py
Automatic merge failed; fix conflicts and then commit the result.

UU data/processed.dvc
UU src/preprocess.py

<<<<<< HEAD
- md5: 557fc53f99bd6aa1b1f1dff9314a99bd.dir
=====
- md5: 9e288ef9f934e89bbef9c66fc299cf22.dir
>>>>>> teammate-sim
```

E4–E5 — Resolution

1. Reconciled both implementations: normalize at float64 precision, then store compact float32 arrays.
2. Regenerated data/processed instead of arbitrarily choosing a stale pointer.
3. Ran `dvc add` and `dvc checkout` to synchronize the resolved DVC object.
4. Committed merge 40120aa, restored pipeline-native output tracking, and reran the pipeline.

```
PS> dvc repro
Stage 'prepare' didn't change, skipping
Running stage 'preprocess' ...
Stage 'train' didn't change, skipping
Running stage 'evaluate' ... test_accuracy: 0.878400

PS> dvc status
Data and pipelines are up to date.
PS> dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'evaluate' didn't change, skipping
```

Submission verification

Deliverable	Evidence	Status
GitHub repository	Full unsquashed history, branches, merge commit, v1/v2 tags	Ready locally
DVC artifacts	Raw/processed/model/history/plot cached; clean dvc status	Ready locally
Google Drive remote	Requires folder ID and one-time OAuth consent; secrets remain local	Authorization pending
PDF report	Git/DVC outputs, configs, metrics, conflict and resolution	Generated

Security note: Google OAuth client ID/secret and cached tokens must never be committed. Configure them with `dvc remote modify --local`; only the remote folder URL is tracked.